UNIVERSIDAD AGRARIA DE COLOMBIA / UNIAGRARIA

Ingeniería Mecatronica

Tutorial de Git y GitHub

Actividad 2 – Configuración de herramientas colaborativas

Jefferson Esteban Restrepo Romero

42274

Carlos Andrés Gómez Vasco

14 de octubre de 2026

Introducción

El presente documento describe el proceso llevado a cabo en la Actividad 2 del curso, la cual tiene como objetivo configurar y utilizar herramientas colaborativas básicas de software usadas en la industria para proyectos de ciencia de datos. A lo largo de este informe se explica qué es Git, cuáles son sus comandos fundamentales, qué es GitHub y cómo se realizó la creación y gestión de un repositorio.

Esta actividad permitió adquirir habilidades prácticas en el uso de Git como sistema de control de versiones y GitHub como plataforma de colaboración, herramientas fundamentales en el ecosistema de la ciencia de datos moderna.

Capítulo 1: ¿Qué es Git?

Git es un sistema de control de versiones distribuido creado por Linus Torvalds en el año 2005. Su propósito principal es registrar los cambios realizados en archivos a lo largo del tiempo, permitiendo a los desarrolladores y científicos de datos llevar un historial detallado de su trabajo.

1.1 Características principales

- Permite ver el historial completo de cambios realizados en un proyecto.
- Facilita volver a versiones anteriores del código o documentos.
- Permite trabajar en equipo de forma organizada y colaborativa.
- Es ampliamente usado en proyectos de software y ciencia de datos.
- Funciona mediante línea de comandos aunque existen interfaces gráficas.

1.2 ¿Para qué sirve Git en ciencia de datos?

En el contexto de la ciencia de datos, Git permite gestionar los cambios en notebooks, scripts de análisis, modelos de machine learning y documentación. Facilita la reproducibilidad de los experimentos y la colaboración entre equipos de investigación.

> **Dato importante**
> Git no es lo mismo que GitHub. Git es el sistema de control de versiones que se instala en el computador local, mientras que GitHub es la plataforma en línea donde se alojan los repositorios de forma remota y colaborativa.

Capítulo 2: Comandos básicos de Git

A continuación se presentan los comandos fundamentales de Git que se utilizaron durante la actividad, junto con una descripción detallada de su función y uso:

2.1 Comandos de configuración e inicio

`git --version` — Verifica que Git esté instalado correctamente.

`git config --global user.name "Nombre"` — Configura el nombre del usuario.

`git config --global user.email "correo"` — Configura el correo del usuario con el que se registro .

`git init` — Inicializa un repositorio nuevo en la carpeta actual.

2.2 Comandos de trabajo diario

`git clone <URL>` — Copia un repositorio remoto a la computadora local.

`git status` — Muestra el estado actual de los archivos.

`git add .` — Agrega todos los archivos modificados al área de preparación.

`git commit -m "mensaje"` — Guarda los cambios con un mensaje descriptivo.

`git push` — Sube los cambios al repositorio remoto en GitHub.

`git pull` — Descarga los cambios del repositorio remoto.

2.3 Flujo de trabajo básico

El flujo de trabajo en Git sigue esta secuencia lógica: primero se realizan cambios en los archivos, luego se agregan con git add, se confirman con git commit describiendo qué se hizo, y finalmente se suben al repositorio remoto con git push. Este ciclo se repite cada vez que se realiza un avance en el proyecto.

Buena práctica

Cada commit debe tener un mensaje claro y descriptivo que explique qué cambio se realizó.

Ejemplo correcto: "Agrego análisis exploratorio de datos de ventas"

Ejemplo incorrecto: "cambios" o "actualización"

Capítulo 3: ¿Qué es GitHub?

GitHub es una plataforma en línea que permite alojar repositorios de Git en la nube. Fue fundada en 2008 y adquirida por Microsoft en 2018. Es actualmente la plataforma de colaboración de código más utilizada en el mundo, con más de 100 millones de usuarios.

3.1 GitHub como portafolio profesional

Como lo destacó el docente Carlos Andrés Gómez Vasco en clase, GitHub no es un disco duro en la nube. Es un portafolio profesional para el ecosistema científico. Un repositorio bien documentado puede ser revisado por empleadores, citado académicamente y utilizado por la comunidad científica.

3.2 Características de GitHub

- No es un disco duro virtual: es un portafolio profesional y herramienta de colaboración.
- Permite trabajar de forma colaborativa con otras personas en el mismo proyecto.
- Los repositorios pueden ser públicos o privados.
- Los proyectos publicados pueden citarse académicamente con DOI.
- Permite crear ramas (branches) para experimentar sin afectar el proyecto principal.
- Facilita la revisión de código mediante Pull Requests.

3.3 Buenas prácticas en GitHub

- Hacer commits descriptivos explicando detalladamente qué se hizo.
- Nunca subir archivos con contraseñas, tokens o credenciales sensibles.
- Documentar siempre el README.md: si el README no explica el proyecto, el proyecto no existe.
- Mantener outputs limpios y organizados en el repositorio.
- Usar comunicación centralizada a través de Issues y comentarios.

Advertencia importante

Nunca subas archivos con contraseñas, llaves API o credenciales a GitHub.
Una vez publicado, cualquier persona puede ver ese dato aunque lo elimines después.
Usa un archivo .gitignore para excluir archivos sensibles del repositorio.

Capítulo 4: Proceso realizado – Informe de actividad

A continuación se describe el proceso paso a paso que se siguió para completar la Actividad 2, incluyendo la instalación de Git, configuración del entorno, creación del repositorio y subida de archivos a GitHub.

Paso 1: Instalación de Git

Se descargó Git desde el sitio oficial git-scm.com versión 2.56.0. Se ejecutó el instalador en Windows seleccionando las opciones por defecto, incluyendo MinTTY como emulador de terminal para Git Bash. Al finalizar la instalación se verificó con el siguiente comando:

```
git --version
```

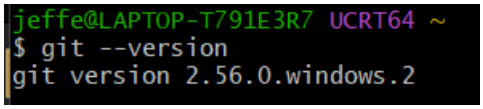


Figura N°1 Resultado obtenido: git version 2.56.0.windows

Paso 2: Configuración de identidad en Git

Se configuró el nombre de usuario y correo electrónico en Git para identificar los commits realizados:

```
git config --global user.name "Jefferson Restrepo"
git config --global user.email "ganadorrgr9@gmail.com"
```

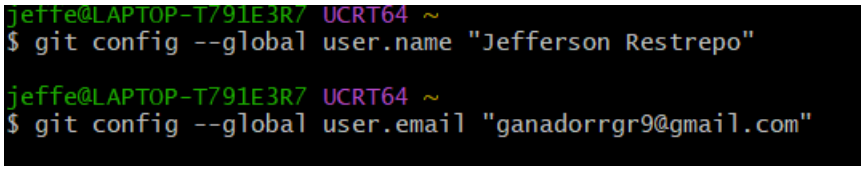


Figura N°2 Resultado obtenido

Paso 3: Creación del repositorio en GitHub

Se creó un repositorio público en GitHub llamado actividad2-git con un archivo README.md inicial. La URL del repositorio quedó disponible en: https://github.com/0Drake/actividad2-git

Paso 4: Clonación del repositorio

Se clonó el repositorio en la computadora local mediante el comando git clone, lo que creó automáticamente la carpeta actividad2-git en el directorio del usuario:

```
git clone https://github.com/0Drake/actividad2-git.git
```

Paso 5: Creación de archivos y subida a GitHub

Se crearon tres archivos de texto con los comandos touch en Git Bash:

```
touch capitulo1.txt
touch capitulo2.txt
```

```
touch capitulo3.txt
```

Se agregó contenido a cada archivo y luego se realizó el proceso completo de add, commit y push:

```
git add .
git commit -m "Agrego los tres capitulos del tutorial de Git"
git push
```

Los archivos quedaron publicados exitosamente en el repositorio de GitHub, verificándose que los tres capítulos aparecen en la plataforma con el mensaje del commit correspondiente.

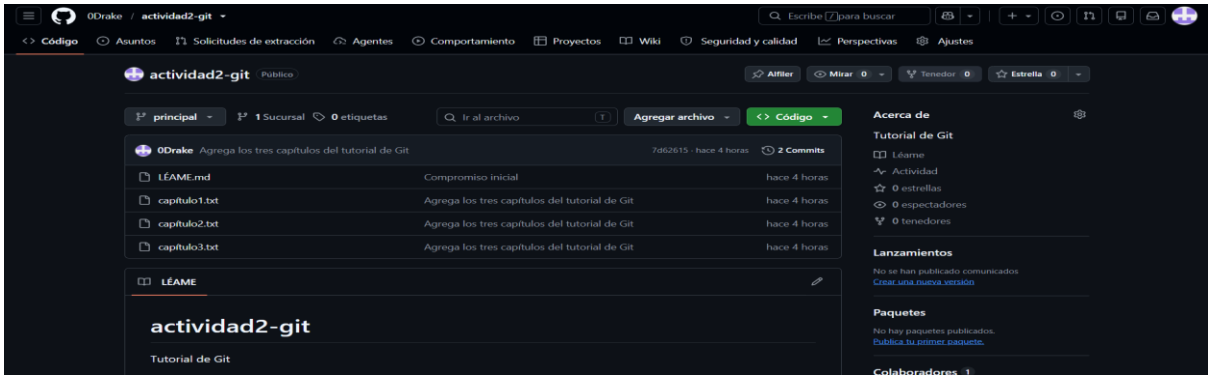


Figura N°3 Resultado obtenido

Dificultades encontradas y soluciones

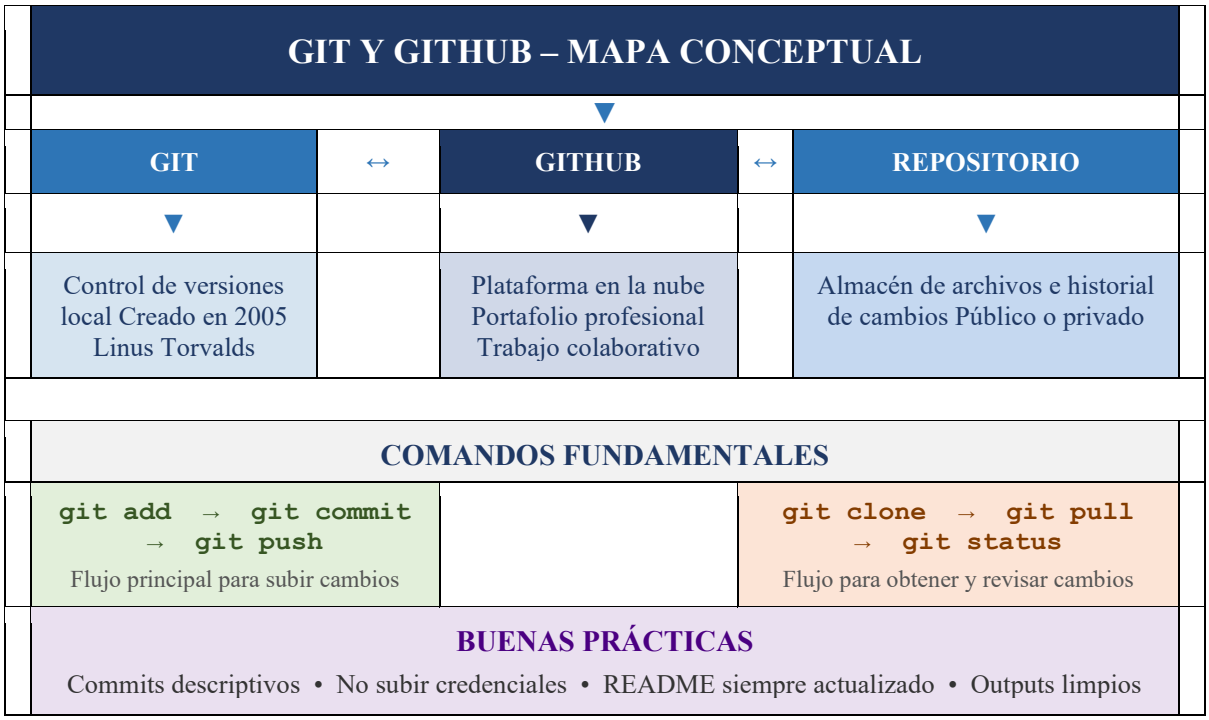
- Al pegar el comando git clone desde el chat, se agregaron caracteres especiales que causaron error. Solución: escribir el comando manualmente en Git Bash.
- Al hacer git push por primera vez, GitHub solicitó autenticación. Solución: iniciar sesión desde el navegador usando la opción Sign in with your browser.

Enlace del repositorio

https://github.com/0Drake/actividad2-git

Mapa Conceptual: Git y GitHub

El siguiente mapa conceptual resume los conceptos principales abordados en este tutorial, mostrando la relación entre Git, GitHub, los repositorios, los comandos fundamentales y las buenas prácticas recomendadas:



Como se observa en el mapa conceptual, Git y GitHub son herramientas complementarias: Git gestiona los cambios localmente y GitHub los aloja de forma remota. Los comandos git add, git commit y git push conforman el flujo principal para registrar y compartir el trabajo, mientras que git clone, git pull y git status permiten obtener y revisar cambios. Todo esto bajo buenas prácticas que garantizan la calidad y seguridad del repositorio.

Referencias

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/en/v2

GitHub, Inc. (2019). GitHub Guides: Learn Git and GitHub (2nd ed.). GitHub. https://guides.github.com/

Hutch Data Science. (n.d.). Using GitHub in a workflow. https://hutchdatascience.org/Tools_for_Reproducible_Workflows_in_R/using-github-in-a-workflow.html

O'Reilly Media. (2020). Git for Teams: A User-Centered Approach to Creating Collaboration. O'Reilly Media.

Van Rossum, G., & Drake, F. L. (2011). Python 3 Reference Manual. CreateSpace Independent Publishing Platform.